

## Problem

A farmer with a fox, a goose, and a sack of corn needs to cross a river. Now he is on the east side of the river and wants to go to west side. The farmer has a rowboat, but there is room for only the farmer and one of his three items. Unfortunately, both the fox and the goose are hungry. The fox cannot be left alone with the goose, or the fox will eat the goose. Likewise, the goose cannot be left alone with the sack of corn, or the goose will eat the corn. Given a sequence of moves find if all the three items fox, goose and corn are safe. The input sequence indicate the item carried by the farmer along with him in the boat. 'F' – Fox, 'C' – Corn, 'G' – Goose, N-Nothing. As he is now on the eastern side the first move is to west and direction alternates for each step.

## Pseudocode

READ items\_carried

SET east as G, C, F

SET west as empty

SET from\_Dir = east and to\_Dir = west

FOR each item in items\_carried

    IF from\_Dir == east THEN

        remove item from east and add to west

        IF east or west contains 'C' and 'G' or 'G' and 'F' THEN

            PRINT 'NOT SAFE'

            BREAK

    ELSE

        remove item from west and add to east

        IF east or west contains 'C' and 'G' or 'G' and 'F' THEN

            PRINT 'NOT SAFE'

            BREAK

    END IF

IF from\_Dir == east THEN

    SET from\_Dir = west

    SET to\_Dir = east

ELSE

    SET from\_Dir = east

    SET to\_Dir = west

END IF

END FOR

PRINT 'SAFE'

While going for a Shopping ....!!!???

Imagine you have the scores in “Python Programming” for 100 students. If you want to find the average score in Python...?

# Simple Statistics

- Many programs deal with large collections of similar information.
  - Words in a document
  - Students in a course
  - Data from an experiment
  - Customers of a business
  - Graphics objects drawn on the screen
  - Cards in a deck

List

# Examples

- Apple, Banana, Berry, Mango
- Football, Basketball, Throwball, Tennis, Hockey
- Sunrise, Sugar, Cheese, Butter, Pickle, Soap, Washing Powder, Oil....
- Agra, Delhi, Kashmir, Jaipur, Kolkata...

# Introduction

- Contains **multiple values** that are **logically related**
- List is a type of **mutable sequence** in Python
- Each element of a list is assigned a number – **index / position**
- Can do indexing, slicing, adding, multiplying, and checking for membership
- Built-in functions for finding **length** of a sequence and for finding its largest and smallest elements

# What is a List?

- Most versatile data type in Python
- Comma-separated items can be collected in square brackets
- Good thing is..
  - THE ITEMS IN THE LIST NEED NOT BE OF SAME TYPE



# Creating a list

- Creating an EMPTY list

```
listname = []
```

Example:

```
L1 = []
```

```
MyList = []
```

```
Books = []
```

- Creating a list with items

```
listname = [item1, item2, ....]
```

Example:

```
Temp = [100, 99.8, 103, 102]
```

```
S = ['15BIT0001', 'Achu',  
     99.9]
```

```
L2 = [1, 2, 3, 4, 5, 6, 7]
```

```
Course = ['Python', 'C', 'C++',  
          'Java']
```

# Accessing Values

- Using index or indices

```
>>>L1 = [1, 2, 3, 4, 5, 6]
```

```
>>>print (L1[3]) #indexing
```

```
>>>4
```

```
>>>print (L1[2:5]) #slicing
```

```
>>>[3, 4, 5]
```

# Updating Elements

- **Update** an element in list using index

```
>>>L1 = [1, 2, 3, 4, 5, 6]
```

```
>>>L1[2] = 111
```

```
>>>L1
```

```
[1, 2, 111, 4, 5, 6]
```

# Deleting Elements

- Delete an element in list using index

```
>>>L1 = [1, 2, 3, 4, 5, 6]
```

```
>>>del (L1[4])
```

```
>>>L1
```

```
[1, 2, 111, 4, 6]
```

# Basic Operations in List

- `>>> len([1, 2, 3])` # Length  
  
3
- `>>> [1, 2, 3] + [4, 5, 6]` # Concatenation  
  
[1, 2, 3, 4, 5, 6]
- `>>> ['Ni!'] * 4` # Repetition  
  
['Ni!', 'Ni!', 'Ni!', 'Ni!']

# Basic Operations in List

- `>>> str([1, 2]) + "34"` # Same as `"[1, 2]" + "34"`  
`'[1, 2]34'`
- `>>> [1, 2] + list("34")`  
# Same as `[1, 2] + ["3", "4"]`  
`[1, 2, '3', '4']`

# List Iteration

- `>>> 3 in [1, 2, 3]` # Membership

`True`

- `>>> for x in [1, 2, 3]:`  
    `print(x, end=' ')`

`# Iteration (2.X uses: print x,) ... 1 2 3`

# List Comprehensions

```
>>> res = [c * 4 for c in 'SPAM']
```

```
# List comprehensions
```

```
>>> res
```

```
['SSSS', 'PPPP', 'AAAA', 'MMMM']
```

- expression is functionally **equivalent** to a **for loop** that builds up a list of results manually
- list comprehensions are **simpler** to code and likely **faster to run** today:



# List Comprehensions

# List comprehension equivalent ...

```
>>> res = []
```

```
>>> for c in 'SPAM':  
    res.append(c * 4)
```

```
>>> res
```

```
['SSSS', 'PPPP', 'AAAA', 'MMMM']
```

# Map

- map built-in function applies a function to items in a sequence and collects all the results in a new list
- `>>> list(map(abs, [-1, -2, 0, 1, 2]))`
- # Map a function across a sequence  
`[1, 2, 0, 1, 2]`

# Indexing, Slicing

```
>>> L = ['spam', 'Spam', 'SPAM!']
```

```
>>> L[2]           # Offsets start at zero
```

```
'SPAM!'
```

```
>>> L[-2]         # Negative: count from the right
```

```
'Spam'
```

```
>>> L[1:]         # Slicing fetches sections
```

```
['Spam', 'SPAM!']
```

# Matrixes

- a basic  $3 \times 3$  two-dimensional list-based array:  
`>>> matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]`
- With **one index**, you get an **entire row** (really, a nested sublist), and with two, you get an item within the row:

```
>>> matrix[1]  
[4, 5, 6]
```

# Matrixes

```
>>> matrix[1][1]
```

```
5
```

```
>>> matrix[2][0]
```

```
7
```

```
>>> matrix = [[1, 2, 3],  
               [4, 5, 6],  
               [7, 8, 9]]
```

```
>>> matrix[1][1]
```

```
5
```

```
>>> L = [1, 2, 3]
```

```
>>> L[1:2] = [4, 5] # Replacement/insertion
```

```
>>> L
```

```
[1, 4, 5, 3]
```

```
>>> L[1:1] = [6, 7] # Insertion (replace nothing)
```

```
>>> L
```

```
[1, 6, 7, 4, 5, 3]
```

```
>>> L[1:2] = [] # Deletion (insert nothing)
```

```
>>> L
```

```
[1, 7, 4, 5, 3]
```

# Insertion, Deletion and Replacement

```
>>> L = [1]
```

```
>>> L[:0] = [2, 3, 4]
```

# Insert all at :0, an empty slice at front

```
>>> L
```

```
[2, 3, 4, 1]
```

```
>>> L[len(L):] = [5, 6, 7]
```

# Insert all at len(L):, an empty slice at end

```
>>> L
```

```
[2, 3, 4, 1, 5, 6, 7]
```

```
>>> L = ['eat', 'more', 'SPAM!']
```

# List method calls

```
>>> L.append('please')
```

# Append method call: add item at end

```
>>> L
```

```
['eat', 'more', 'SPAM!', 'please']
```

```
>>> L.sort()
```

# Sort list items ('S' < 'e')

```
>>> L
```

```
['SPAM!', 'eat', 'more', 'please']
```



# More on Sorting Lists

```
>>> L = ['abc', 'ABD', 'aBe']
```

```
>>> L.sort() # Sort with mixed case
```

```
>>> L
```

```
['ABD', 'aBe', 'abc']
```

```
>>> L = ['abc', 'ABD', 'aBe']
```

```
>>> L.sort(key=str.lower) # Normalize to lowercase
```

```
>>> L
```

```
['abc', 'ABD', 'aBe']
```

# More on Sorting Lists

```
>>> L.sort(key=str.lower, reverse=True)
```

# Change sort order

```
>>> L ['aBe', 'ABD', 'abc']
```

# Other common list methods

```
>>> L = [1, 2]
```

```
>>> L.extend([3, 4, 5])
```

# Add many items at end (like in-place +)

```
>>> L [1, 2, 3, 4, 5]
```

```
>>> L.pop()
```

# Delete and return last item

```
5
```

# Other common list methods

```
>>> L
```

```
[1, 2, 3, 4]
```

```
>>> L.reverse() # In-place reversal method
```

```
>>> L
```

```
[4, 3, 2, 1]
```

```
>>> list(reversed(L)) # Reversal built-in with a result  
      (iterator)
```

```
[1, 2, 3, 4]
```

# Other common list methods

```
>>> L = ['spam', 'eggs', 'ham']
```

```
>>> L.index('eggs')    # Index of an object (search/find)
```

```
1
```

```
>>> L.insert(1, 'toast') # Insert at position
```

```
>>> L
```

```
['spam', 'toast', 'eggs', 'ham']
```

```
>>> L.remove('eggs')    # Delete by value
```

```
>>> L
```

```
['spam', 'toast', 'ham']
```

# Other common list methods

```
>>> L.pop(1)                # Delete by position 'toast'
```

```
>>> L
```

```
['spam', 'ham']
```

```
>>> L.count('spam')        # Number of occurrences 1
```

```
1
```

# Other common list methods

```
>>> L = ['spam', 'eggs', 'ham', 'toast']
```

```
>>> del L[0]                                # Delete one item
```

```
>>> L
```

```
['eggs', 'ham', 'toast']
```

```
>>> del L[1:]                                # Delete an entire section
```

```
>>> L                                         # Same as L[1:] = []
```

```
['eggs']
```

# Other common list methods

```
>>> L = ['Already', 'got', 'one']
```

```
>>> L[1:] = []
```

```
>>> L
```

```
['Already']
```

```
>>> L[0] = []
```

```
>>> L
```

```
[[]]
```



# Hence...

- A list is a sequence of items stored as a single object.
- Items in a list can be accessed by indexing, and sub lists can be accessed by slicing.
- Lists are mutable; individual items or entire slices can be replaced through assignment statements.
- Lists support a number of convenient and frequently used methods.
- Lists will grow and shrink as needed.

# Strings and Lists

```
>>> S = 'spammy'
```

```
>>> L = list(S)
```

```
>>> L
```

```
['s', 'p', 'a', 'm', 'm', 'y']
```

```
>>> L[3] = 'x'
```

# Works for lists, not strings

```
>>> L[4] = 'x'
```

```
>>> L
```

```
['s', 'p', 'a', 'x', 'x', 'y']
```

```
>>> S = ''.join(L)    #uses " for joining elements of list
```

```
>>> S
```

```
'spaxxy'
```

# Strings and Lists

```
>>> 'SPAM'.join(['eggs', 'sausage', 'ham', 'toast'])
```

```
'eggsSPAMsausageSPAMhamSPAMtoast'
```

# uses 'SPAM' for joining elements of list

```
>>> line = 'aaa bbb ccc'
```

```
>>> cols = line.split()
```

```
>>> cols
```

```
['aaa', 'bbb', 'ccc']
```

# Statistics using List

Find the mean, standard deviation and median  
of a set of numbers

Table 8-1. Common list literals and operations

| Operation                                      | Interpretation                                           |
|------------------------------------------------|----------------------------------------------------------|
| <code>L = []</code>                            | An empty list                                            |
| <code>L = [123, 'abc', 1.23, {}]</code>        | Four items: indexes 0..3                                 |
| <code>L = ['Bob', 40.0, ['dev', 'mgr']]</code> | Nested sublists                                          |
| <code>L = list('spam')</code>                  | List of an iterable's items, list of successive integers |
| <code>L = list(range(-4, 4))</code>            |                                                          |
| <code>L[i]</code>                              | Index, index of index, slice, length                     |
| <code>L[i][j]</code>                           |                                                          |
| <code>L[i:j]</code>                            |                                                          |
| <code>len(L)</code>                            |                                                          |
| <code>L1 + L2</code>                           | Concatenate, repeat                                      |

| Operation                                 | Interpretation                                                                                                       |
|-------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| <code>L * 3</code>                        |                                                                                                                      |
| <code>for x in L: print(x)</code>         | Iteration, membership                                                                                                |
| <code>3 in L</code>                       |                                                                                                                      |
| <code>L.append(4)</code>                  | Methods: growing                                                                                                     |
| <code>L.extend([5,6,7])</code>            |                                                                                                                      |
| <code>L.insert(i, X)</code>               |                                                                                                                      |
| <code>L.index(X)</code>                   | Methods: searching                                                                                                   |
| <code>L.count(X)</code>                   |                                                                                                                      |
| <code>L.sort()</code>                     | Methods: sorting, reversing,                                                                                         |
| <code>L.reverse()</code>                  | copying (3.3+), clearing (3.3+)                                                                                      |
| <code>L.copy()</code>                     |                                                                                                                      |
| <code>L.clear()</code>                    |                                                                                                                      |
| <code>L.pop(i)</code>                     | Methods, statements: shrinking                                                                                       |
| <code>L.remove(X)</code>                  |                                                                                                                      |
| <code>del L[i]</code>                     |                                                                                                                      |
| <code>del L[i:j]</code>                   |                                                                                                                      |
| <code>L[i:j] = []</code>                  |                                                                                                                      |
| <code>L[i] = 3</code>                     | Index assignment, slice assignment                                                                                   |
| <code>L[i:j] = [4,5,6]</code>             |                                                                                                                      |
| <code>L = [x**2 for x in range(5)]</code> | List comprehensions and maps ( <a href="#">Chapter 4</a> , <a href="#">Chapter 14</a> , <a href="#">Chapter 20</a> ) |
| <code>list(map(ord, 'spam'))</code>       |                                                                                                                      |

# Exercise 1

Given a positions of coins of player1 and player2 in a 3X3 Tic Tac Toc board, write a program to determine if the board position is a solution and if so identify the winner of the game. In a Tic Tac Toc problem, if the coins in a row or column or along a diagonal is of the same player then he has won the game. Assume that player1 uses '1' as his coin and player2 uses '2' as his coin. '0' in the board represent empty cell.

## Exercise 2

- In a supermarket there are two sections S1 and S2. The sales details of item<sub>1</sub> to item<sub>n</sub> of section1 and item<sub>1</sub> to item<sub>p</sub> of section2 are maintained in a sorted order. Write a program to merge the elements of the two sorted lists to form the consolidated list.



## Exercise 3

- Watson gives Sherlock an list of  $N$  *numbers*. Then he asks him to determine if there exists an element in the list such that the sum of the elements on its left is equal to the sum of the elements on its right. If there are no elements to the left/right, then the sum is considered to be zero.

## Exercise 4

- Sunny and Johnny together have  $M$  dollars they want to spend on ice cream. The parlor offers  $N$  flavors, and they want to choose two flavors so that they end up spending the whole amount.
- You are given the cost of these flavors. The cost of the  $i$ th flavor is denoted by  $c_i$ . You have to display the indices of the two flavors whose sum is  $M$ .

## Exercise 5

- Given a list of integer values, find the fraction of count of positive numbers, negative numbers and zeroes to the total numbers. Print the value of the fractions correct to 3 decimal places.

## Exercise 6

- Given  $N$  integers, count the number of pairs of integers whose difference is  $K$ .